Institute of Technology of Cambodia

Department of Electrical and Energy Engineering

Génie
Electrique et Energétique

Computer Programming Final Project:

Zener Diode Voltage Regulator Calculator.

Lab Report by **I3-GEE-C1, Team 4.**

1. **TANG MENGSONG** **ID**: e20220271
2. **THAI TAIWAN** **ID**: e20220413
3. **THON SAKSAMBO** **ID**: e20240038
4. **TANN MENGHAK** **ID**: e20221590
5. **TAING HYMENG** **ID**: e20220644

Date: 07 February 2025

Objective: Calculate the key parameters of a Zener diode voltage regulator circuit, including output voltage, load current, series resistor current, and power dissipated by the Zener diodes.

Contents

1 Introduction

1.1 Background of the Problem

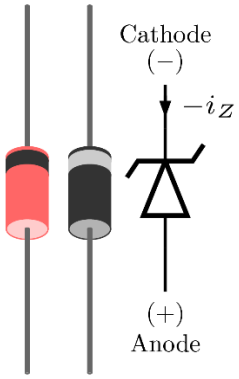


Figure 1.1 Zener Diode [1]

There is a need for stable and reliable power supply, voltage source for proper functioning of some electrical components. Voltage regulation is a method to make sure that the voltage is stable for some loads, and Zener diodes are widely used for this purpose due to their **simplicity** and **effectiveness**. By operating in their reverse breakdown mode, Zener diodes maintain a constant output voltage, even if there's variations in the input voltage or load conditions.

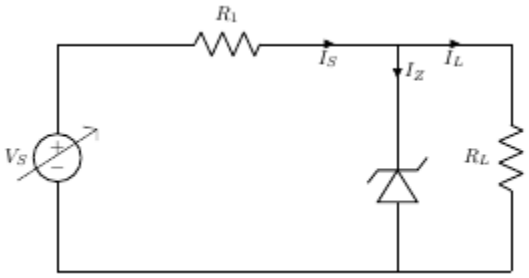


Figure 1.2 Zener diode line regulation [1]

A voltage regulator is an electronic circuit that provides a stable DC voltage independent of the load current, temperature and AC line voltage variations. A Zener diode of break down voltage VZ is reverse connected to an input voltage source VI across a load resistance RL and a series resistor RS. The voltage across the zener will remain steady at its break down voltage VZ for all the values of zener current IZ as long as the current remains in the break down region. Hence a regulated DC output voltage V0=VZ is obtained across RL, whenever the input voltage remains within a minimum and maximum voltage. Basically there are two type of regulations such as: Line Regulation: In this type of regulation, series resistance and load resistance are fixed, only input voltage is changing. Output voltage remains the same as long as the input voltage is maintained above a minimum value. Load Regulation: In this type of regulation, input voltage is fixed and the load resistance is varying. Output volt remains same, as long as the load resistance is maintained above a minimum value. [1]

1.2 Rationale for Selecting the Problem

Voltage regulation using Zener diodes is fundamental in electronics. By selecting this problem, the project can achieve:

1. Demonstration of application of Zener diodes in practical scenarios.
2. Can make users simulate and analyze different conditions, such as supply voltage, series resistance, and load resistance.
3. Narrow the gap between theoretical understanding and practical circuit design through program simulation and calculations.

2 Objective of Final Project

2.1 Objective

Calculate the key parameters of a Zener diode voltage regulator circuit, including output voltage, load current, series resistor current, and power dissipated by the Zener diodes.

2.2 Scope

This project simulates the behavior of a Zener diode voltage regulator using a C program. It calculates key circuit parameters such as **output voltage**, **load current**, **series resistor current**, and **power dissipation**. The program makes sure that the user interface is friendly by putting input validation, error handling, and the ability to perform multiple calculations in one session. While focused on simulation, the project also include some physical implementation and real-world factors like power effects or real component tolerances.

3 Methodology

3.1 Materials

-Hardware used: A running computer with C compiler installed, a text editor or an IDE.

-Software used: Visual Studio Code, MinGW, GNU GCC.

3.2 Final Project Algorithms

3.2.1 main.c

Algorithm Steps:

1. Start the program.
2. Declare a variable **choice** as **char** datatype to store in the user's choice whether to continue or add another calculation.
3. Start with do from do-while loop.
4. Call the function **userParameters()** (defined in **zener.c**)
5. Ask if the user wants to see previous calculations.
6. Clear input buffer.
7. Read user input and decide whether to show or not by calling viewPreviousResults() function.
8. Show the message "Do you want to perform another calculation (y/n)?" to the user.

9. Clear input buffer.
10. Read and store the user's input into the **choice** variable.
11. Now in **while,** check the condition:
    -If the input is "**y**" or "**Y**" then while is true and continue the loop.
    -If the input is neither of these then **end** the **do-while loop**.
12. Show the message: "Thank you for using the Zener diode calculator. Goodbye!"
13. **End the program**.

In **viewPreviousResults():**

1. Open the file zener_results.txt in read mode.
2. If the file does not exist, print "No previous results found." and return.
3. Print a header: "===== Previous Calculations =====".
4. Read and print the file contents character by character:
5. Use fgetc(file) to read a character.
6. Use putchar(ch) to print the character.
7. Repeat until EOF (End of File) is reached.
8. Close the file.
9. Print a footer: "==========".
10. Return to the main() function.

3.2.2 zener.c

There are two functions in zener.c, the first function to be called is:

userParameters():

1. Start the function.
2. Initialize variables **Vin**, **Vz**, **Rs**, **RL**, **Prs**, **Pz_max** to store important electrical parameters. And **result** to store the return value of **scanf()**;
3. Start an infinite while loop and ask user repeatedly **until valid values are provided**.
4. Prompt the user to enter each electrical parameter and check if each input is valid or not.
   -If it's a negative number or not a number then display error and clear the input buffer.
   -If it's valid then move on to the next parameter.
5. If all inputs are valid then **call** another function in **zener.c** and pass on the parameters as arguments to it.
6. Exit the loop by break.

calculateZenerParameters():

1. Start the function.
2. Initialize variables like **Vout**, **Iout**, **Pz**, **Prs_dissipated** to store calculated values.
3. Open a **file** "Zener_results.txt" in write mode to store results.
4. Check if **files** are opened successfully: if fail display and error and return.
5. Check if the Zener is in breakdown mode:
   -If Vin < Vz then show a warning message and log it to the file.
   -Exit the function early.

6. Perform calculations (only when **Vin >= Vz**)
    - Set **Vout = Vz**: regulated voltage equals to Zener breakdown voltage
    - **Iout = Vout / RL**: to calculate output current
    - **Irs = (Vin - Vout)** / **Rs**: calculate current through series resistance
    - **Prs_dissipated = (Irs^2) * Rs: power** dissipated in series resistor
    - **Pz = vout * (Irs – Iout):** Power dissipated in Zener diode.
7. Check for unsafe conditions:
    - -If **Irs > 1.0A** show and log warning that the series resistance is high.
    - -If **Iout > (100.0/RL) A**: show and log warning of high load current.
    - -If **Prs_dissipated > Prs**: Print and log warning of (Excessive power dissipation in **Rs**)
    - -If **Pz > Pz_max**: Print and log warning of overheating of Zener diode.
8. Display results and log into file: **Vout**, **Iout**, **Irs**, **Prs_dissipated**, Pz in **milli** units.
9. Close the file and exit the function.

3.2.3 zener.h

- Declare function void userparameters():
- Declare function void calculateZenerParameters(double Vin, double Vz, double Rs, double RL, double Prs, double Pz_max);

3.3 Final Project Flowchart

3.3.1 main.c

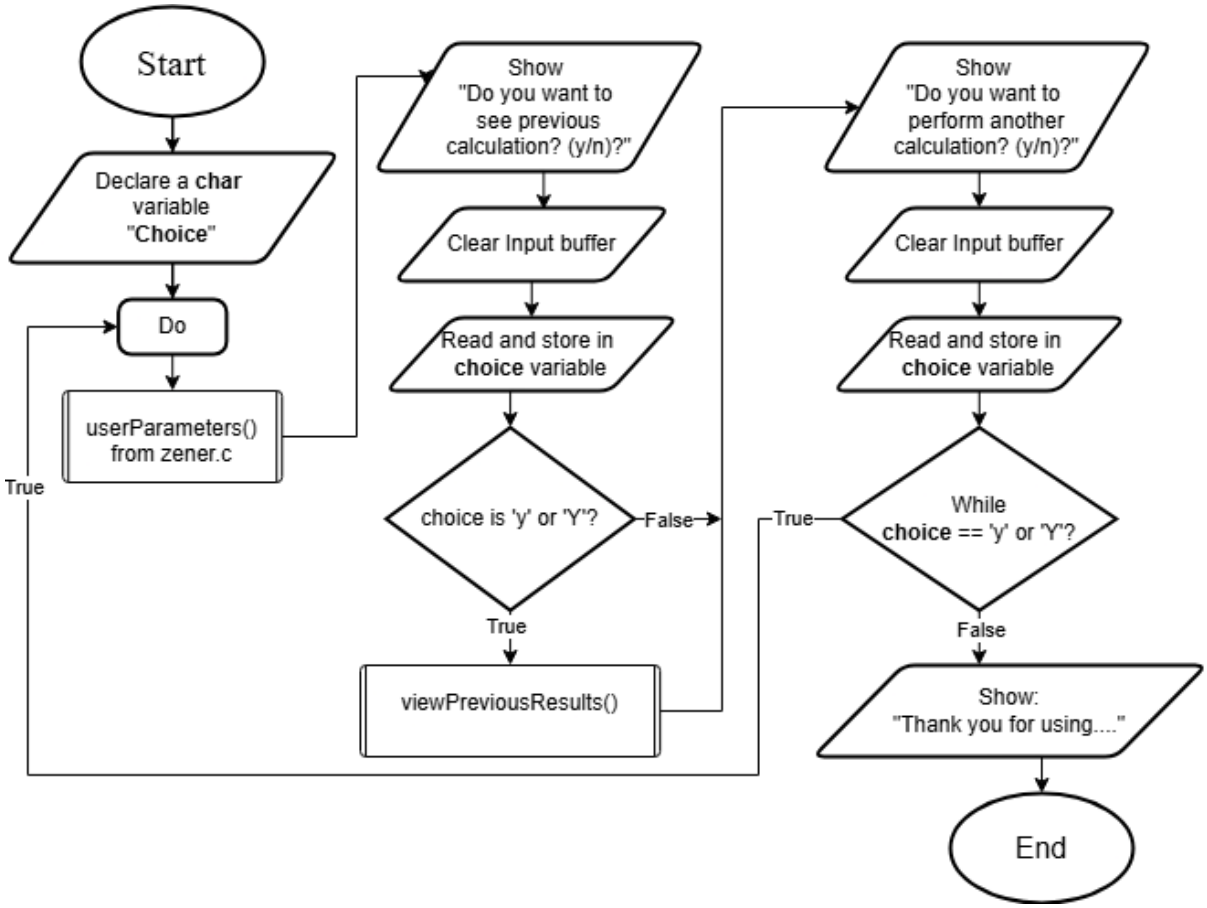
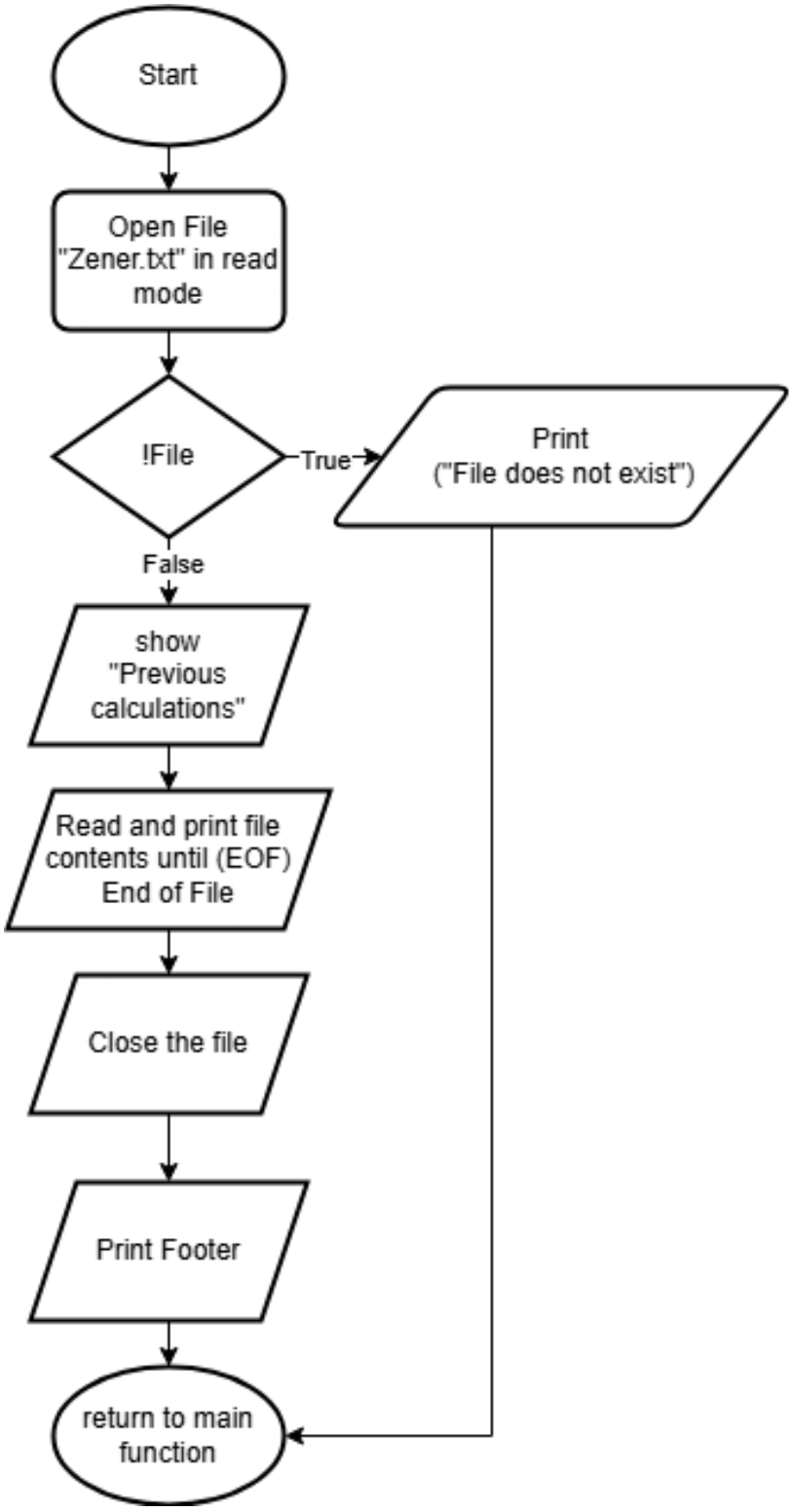


Figure 3 Flowchart for main.c

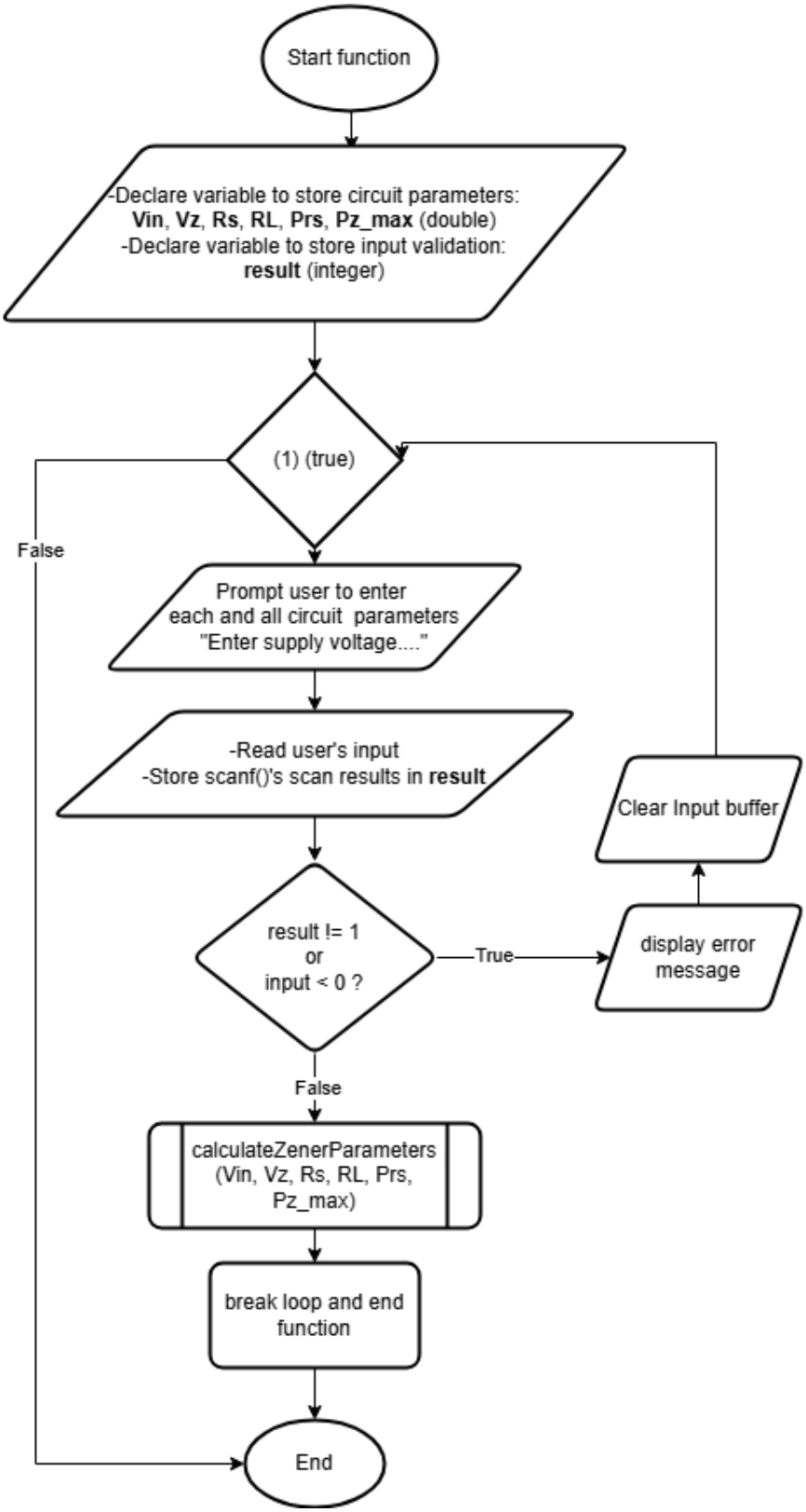
viewPreviousResults():



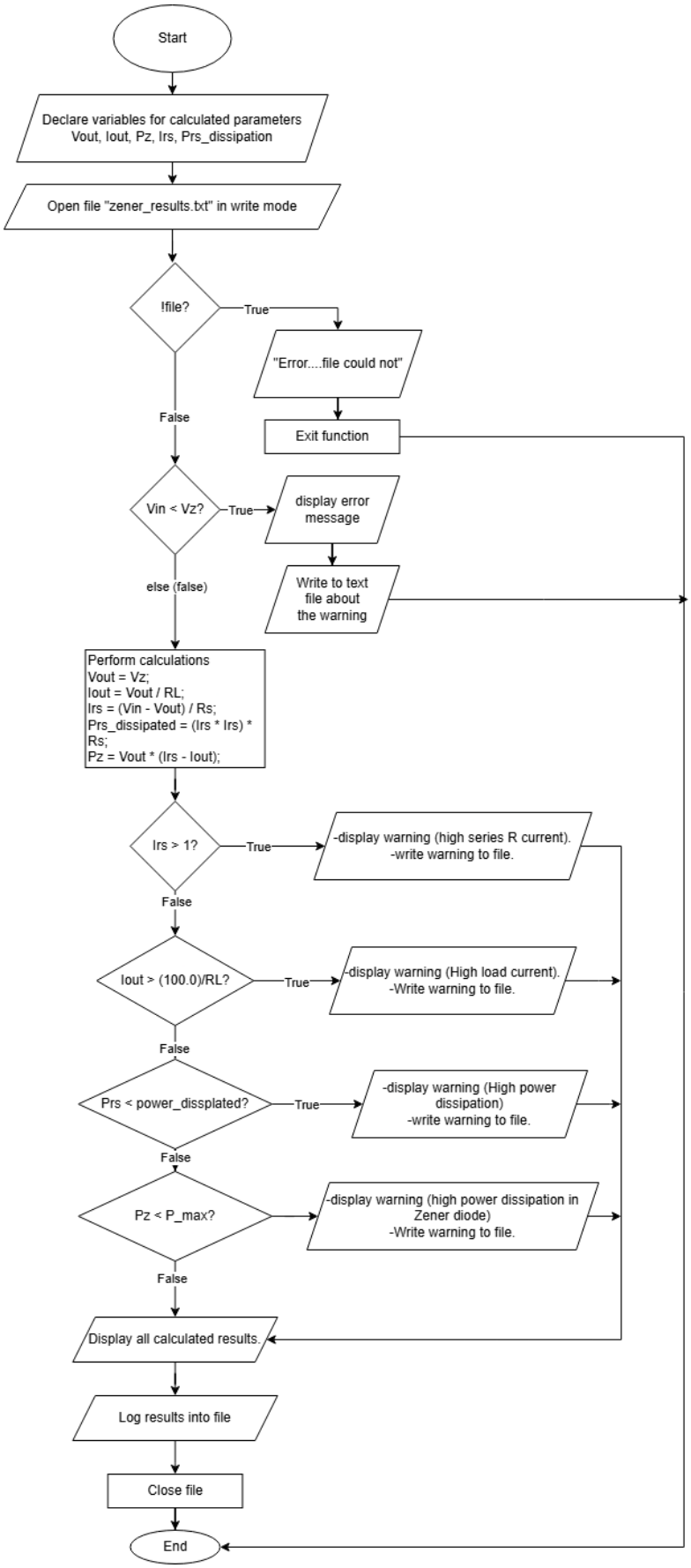
3.3.2 zener.c

In zener.c there are two **functions:**

userParameters() to prompt user, to validate inputs and to pass on values :



calculateZenerParameters() to do necessary calculations for our Zener voltage regulators:



3.4 Final Project C Program to Implement the Solution:

main.c;

```
#include <stdio.h>
#include <stdlib.h>

#include "zener.h"

void viewPreviousResults() {
    FILE *file = fopen("zener_results.txt", "r"); // Open the file in read mode

    if (!file) { // Check if the file exists
        printf("No previous results found.\n"); // Display message if file is missing
        return; // Exit function
    }

    printf("\n===== Previous Calculations =====\n");

    char ch;
    // Read and print the file character by character until end of file
    while ((ch = fgetc(file)) != EOF) {
        putchar(ch); // Print each character to the console
    }
    fclose(file); // Close the file after reading
    printf("\n==================================\n");
}

int main() {
    char choice; // Variable to store user input for repeating the program

    do {
        userParameters(); // Call function to take user input and perform calculations

        // Ask the user if they want to view previous calculations
        printf("\nDo you want to view previous calculations? (y/n): ");
        while (getchar() != '\n'); // Clear the input buffer to avoid reading previous
inputs
        scanf(" %c", &choice); // Read user's choice

        if (choice == 'y' || choice == 'Y') { // If user enters 'y' or 'Y'
            viewPreviousResults(); // Display previous calculations
        }

        // Ask the user if they want to perform another calculation
        printf("\nDo you want to perform another calculation? (y/n): ");
        while (getchar() != '\n'); // Clear input buffer again
        scanf(" %c", &choice); // Read user's choice

    } while (choice == 'y' || choice == 'Y'); // Loop again if user wants another
calculation

    printf("Thank you for using the Zener diode calculator. Goodbye!\n");

    return 0;
}
```

zener.h:

```
#ifndef ZENER_H
#define ZENER_H

void userParameters();
```

```
void calculateZenerParameters(double Vin, double Vz, double Rs, double RL, double Prs,
double Pz_max);

#endif
```

zener.c

```
#include <stdio.h>  // Standard input-output library for printf, scanf, etc.
#include <stdlib.h> // Standard library for system functions like memory allocation

#include "zener.h"  // Custom header file that contains function declarations

void userParameters() {
    double Vin, Vz, Rs, RL, Prs, Pz_max; // Circuit parameters
    int result; // Variable to store input validation result

    // Infinite loop to ensure valid input is received before proceeding
    while (1) {
        // Prompting user for circuit parameters
        printf("Enter supply voltage (Vin) in volts: ");
        result = scanf("%lf", &Vin);
        if (result != 1 || Vin < 0) { // Validate input
            printf("\nERROR: Invalid input! Supply voltage (Vin) must be a non-negative
number.\n");
            while (getchar() != '\n'); // Clear input buffer
            continue; // Restart loop
        }

        printf("Enter Zener breakdown voltage (Vz) in volts: ");
        result = scanf("%lf", &Vz);
        if (result != 1 || Vz < 0) {
            printf("\nERROR: Invalid input! Zener breakdown voltage (Vz) must be a non-
negative number.\n");
            while (getchar() != '\n');
            continue;
        }

        printf("Enter series resistance (Rs) in ohms: ");
        result = scanf("%lf", &Rs);
        if (result != 1 || Rs < 0) {
            printf("\nERROR: Invalid input! Series resistance (Rs) must be a non-negative
number.\n");
            while (getchar() != '\n');
            continue;
        }

        printf("Enter series resistor power rating (Prs) in watts: ");
        result = scanf("%lf", &Prs);
        if (result != 1 || Prs < 0) {
            printf("\nERROR: Invalid input! Series resistor power rating must be a non-
negative number.\n");
            while (getchar() != '\n');
            continue;
        }

        printf("Enter load resistance (RL) in ohms: ");
        result = scanf("%lf", &RL);
        if (result != 1 || RL < 0) {
            printf("\nERROR: Invalid input! Load resistance (RL) must be a non-negative
number.\n");
            while (getchar() != '\n');
            continue;
        }

        printf("Enter Zener diode power rating (Pz_max) in watts: ");
```

```
        result = scanf("%lf", &Pz_max);
        if (result != 1 || Pz_max < 0) {
            printf("\nERROR: Invalid input! Zener diode power rating must be a non-negative
number.\n");
            while (getchar() != '\n');
            continue;
        }

        // Call function to calculate and display circuit parameters
        calculateZenerParameters(Vin, Vz, Rs, RL, Prs, Pz_max);
        break; // Exit loop after successful input
    }
}

void calculateZenerParameters(double Vin, double Vz, double Rs, double RL, double Prs,
double Pz_max) {
    double Vout, Iout, Pz, Irs, Prs_dissipated; // Calculated parameters
    FILE *file = fopen("zener_results.txt", "a"); // Open file in append mode

    if (!file) { // Check if file opened successfully
        printf("ERROR: Could not open file for writing.\n");
        return;
    }

    // Check if the Zener diode is in breakdown mode
    if (Vin < Vz) {
        printf("\n=====================================================\n");
        printf("WARNING: The Zener diode is NOT in breakdown mode.\n");
        printf("Output Voltage (Vout): %.2f V (equals supply voltage as no regulation is
occurring)\n", Vin);
        printf("=====================================================\n\n");

        // Log warning to file
        fprintf(file, "WARNING: The Zener diode is NOT in breakdown mode.\n");
        fprintf(file, "Output Voltage (Vout): %.2f V (equals supply voltage)\n\n", Vin);
//after printing, there's nth to do so it exit prematurely
    } else {
        // Perform calculations
        Vout = Vz;                    // Output voltage remains at Zener voltage
        Iout = Vout / RL;              // Load current calculation (Ohm's Law)
        Irs = (Vin - Vout) / Rs;       // Current through series resistor
        Prs_dissipated = (Irs * Irs) * Rs; // Power dissipated in series resistor
        Pz = Vout * (Irs - Iout);      // Power dissipated in the Zener diode

        // Print and log warnings based on safety limits
        if (Irs > 1.0) {
            printf("\nWARNING: Excessive current through series resistor!\n");
            fprintf(file, "WARNING: Excessive current through series resistor!\n");
        }
        if (Iout > (100.0 / RL)) {
            printf("\nWARNING: High load current detected!\n");
            fprintf(file, "WARNING: High load current detected!\n");
        }
        if (Prs_dissipated > Prs) {
            printf("\nWARNING: Excessive power dissipation in Series Resistor!\n");
            fprintf(file, "WARNING: Excessive power dissipation in Series Resistor!\n");
        }
        if (Pz > Pz_max) {
            printf("\nWARNING: High power dissipation in Zener diode!\n");
            fprintf(file, "WARNING: High power dissipation in Zener diode!\n");
        }

        // Display calculated results
        printf("\n=== Zener Diode Voltage Regulator Results ===\n");
        printf("Output Voltage (Vout): %.2f V\n", Vout);
        printf("Load Current (Iout): %.2f mA\n", Iout * 1000);
```

```
        printf("Current Through Series Resistor (Irs): %.2f mA\n", Irs * 1000);
        printf("Power Dissipated by Series Resistor (Prs): %.2f mW\n", Prs_dissipated *
1000);
        printf("Power Dissipated by Zener Diode (Pz): %.2f mW\n", Pz * 1000);

        // Log results to file
        fprintf(file, "\n=== Zener Diode Voltage Regulator Results ===\n");
        fprintf(file, "Output Voltage (Vout): %.2f V\n", Vout);
        fprintf(file, "Load Current (Iout): %.2f mA\n", Iout * 1000);
        fprintf(file, "Current Through Series Resistor (Irs): %.2f mA\n", Irs * 1000);
        fprintf(file, "Power Dissipated by Series Resistor (Prs): %.2f mW\n",
Prs_dissipated * 1000);
        fprintf(file, "Power Dissipated by Zener Diode (Pz): %.2f mW\n\n", Pz * 1000);
    }

    fclose(file); // Close file after writing }
```

3.5 Challenges faced

- **Input Validation**: Making sure that the program can handle invalid inputs, like non-numeric characters or negative numbers. Solutions like clearing the input buffer and checking scanf()'s return value were used to solve this issue.
- **Algorithm Design**: Making a clear, step-by-step algorithm for the calculation loop, including handling cases like Vin < Vz, is hard.
- **User Experience**: Making sure that the program is user-friendly with prompts, it outputs error messages, and the ability to perform multiple calculations was challenging.

3.6 Changes from Mini Project

- Improved input warnings
- Added electrical warnings
- Fixed loop not restarting
- Separated programs into library and header files for modularity
- Improved algorithms and more modularity.
- Taking more element variables into consideration.

4 Results

- Under normal calculations where no inputs are invalid

```
Enter supply voltage (Vin) in volts: 12
Enter Zener breakdown voltage (Vz) in volts: 5
Enter series resistance (Rs) in ohms: 10000
Enter series resistor power rating (Prs) in watts: 0.5
Enter load resistance (RL) in ohms: 300
Enter Zener diode power rating (Pz_max) in watts: 1

=== Zener Diode Voltage Regulator Results ===
Output Voltage (Vout): 5.00 V
Load Current (Iout): 16.67 mA
Current Through Series Resistor (Irs): 0.70 mA
Power Dissipated by Series Resistor (Prs): 4.90 mW
Power Dissipated by Zener Diode (Pz): -79.83 mW

Do you want to view previous calculations? (y/n):
```

Figure 4.1 Normal Operation.

- Seeing previous calculations:

```
Do you want to view previous calculations? (y/n): y

===== Previous Calculations =====

=== Zener Diode Voltage Regulator Results ===
Output Voltage (Vout): 10.00 V
Load Current (Iout): 3.33 mA
Current Through Series Resistor (Irs): 0.20 mA
Power Dissipated by Series Resistor (Prs): 0.40 mW
Power Dissipated by Zener Diode (Pz): -31.33 mW


=== Zener Diode Voltage Regulator Results ===
Output Voltage (Vout): 5.00 V
Load Current (Iout): 16.67 mA
Current Through Series Resistor (Irs): 0.70 mA
Power Dissipated by Series Resistor (Prs): 4.90 mW
Power Dissipated by Zener Diode (Pz): -79.83 mW
```

- When one of the input numbers is invalid but the program restarts while loop

```
Enter supply voltage (Vin) in volts: 12
Enter Zener breakdown voltage (Vz) in volts: -10

ERROR: Invalid input! Zener breakdown voltage (Vz) must be a non-negative number.
Enter supply voltage (Vin) in volts: 
```

Figure 4.2 One of the input numbers is invalid.

- When one of the input data types is invalid

```
Enter supply voltage (Vin) in volts: 12
Enter Zener breakdown voltage (Vz) in volts: 3
Enter series resistance (Rs) in ohms: f

ERROR: Invalid input! Series resistance (Rs) must be a non-negative number.
Enter supply voltage (Vin) in volts: 
```

Figure 4.3 One of the input's data types is invalid.

- When Vin < Vz

```
Enter supply voltage (Vin) in volts: 1
Enter Zener breakdown voltage (Vz) in volts: 12
Enter series resistance (Rs) in ohms: 10000
Enter series resistor power rating (Prs) in watts: 0.5
Enter load resistance (RL) in ohms: 500
Enter Zener diode power rating (Pz_max) in watts: 1

==================================================
WARNING: The Zener diode is NOT in breakdown mode.
Output Voltage (Vout): 1.00 V (equals supply voltage as no regulation is occurring)
==================================================


Do you want to view previous calculations? (y/n): 
```

Figure 4.4 Program's output when Vin < Vz

5 Discussion

-The program can handle invalid inputs and prevent unexpected crashes.

+The program can show the Zener Diode behavior by:

-Show proper voltage regulation only when **Vin** >= **Vz**.

-Shows correct handling of non-breakdown mode when **Vin** < **Vz**.

-Shows when excessive currents are happening on elements.

+The program can restart loops in case of wrong inputs.

+The program has user-friendly interface with clear prompts and outputs simplifies the simulation of Zener diode voltage regulation.

6 GitHub

QR code and **Link** for **GitHub Repository** for this project:

https://github.com/sakSambo/Final-Project

7 References

[1] C. B. 3. h. By Teravolt (talk) (Transferred by Nk/Originally uploaded by Teravolt) - I (Teravolt (talk)) created this work entirely by myself. (Originally uploaded on en.wikipedia), "Zener Diode," [Online]. Available: https://en.wikipedia.org/wiki/Zener_diode.

[2] Virtual Labs, "Zener Diode-Voltage Regulator," [Online]. Available: https://be-iitkgp.vlabs.ac.in/exp/voltage-regulator/theory.html.